# GOLD: Using Objects (Fri Sep 18)

## Introduction

### Introducing Objects

Consider these lines of code:

```
s.length()
list.add("cheese")
Math.pow(2, 5)
new Random()
```

- What is `new` doing?
- What is the `.` doing?
- Which lines name a "kind of thing" versus an actual thing?

# Objects and Classes

## Using Objects and Classes

Everything beyond numbers, booleans, and characters in the last month has been an ***object*** or a ***class***:

| You wrote | It really is |
|---|---|
| `String name = "Boiler";`<br>`name.length()` | a `String` **object** and `length()` is one of its **methods** |
| `int[] a = new int[5];`<br>or<br>`ArrayList<Integer> xs = ...` | array / `ArrayList` **objects** that hold many values |
| `Math.sqrt(2)`, `Math.PI` | the `Math` **class.**<br>Contains methods and a constant that you call directly |
| `Integer.parseInt("42")` | a **method** of the `Integer` **class** |

Today,

- we'll look closer at specific useful classes and
- we will learn what's going on in the syntax above and how to describe it precisely

## Vocabulary: class, object, field, method

- A **class** is a *type.* Think of it as a blueprint.
  - `String`,
  - `Random`,
  - `ArrayList`,
  - `Math` are all classes.
- An **object** (or **instance**) is a concrete thing made from a class.
  - `new String("CS 180")` makes one `String` object.
  - `new Random()` makes one `Random` object.

- A **field** is a piece of data inside an object (or class):
  - `a.length`,
  - `Math.PI`.

- A **method** is an *operation* you can ask for:
  - `s.length()`,
  - `list.add(x)`,
  - `Double.parseDouble("3.14159")`,
  - `Math.pow(2, 5)`.

- We refer to both the fields and the methods when we say **members** of a class

# Static and Non-static

## Fields and methods come in two flavors

Members of a class can be **static** or **non-static** (also called **instance**):

- **non-static ("of the object")**
  - belongs to each individual object
  - you need an object to use it.
  - Examples: `s.length()`, `list.add(x)`.
  - Think why: it would not make sense to write ~~String.length()~~

- **static ("of the class")**
  - belongs to the class itself
  - no object needed.
  - Examples: `Math.pow(...)`, `Integer.parseInt(...)`, `Math.PI`.

We've used both already!

## Two ways to call a method

**On the class** (static). No object needed, so we can just use the class name:

```
double p = Math.pow(2, 5);          // 32.0
int    n = Integer.parseInt("42");  // 42
```

**On an object** (instance or non-static). We need an object first (a literal or a `new`):

```
String s = "Boiler";
int len = s.length();               // s is the object

var list = new ArrayList<String>();
list.add("cheese");                 // list is the object
```

Remember: **`ClassName.method(...)`** vs **`object.method(...)`**.

- This is why we emphasize Capital Letters for class names and lowercase for variables!

**Our code so far**: we've been calling methods we write without an object or a class

- There is an object....
  - Every file we write is one class
  - The running program is an object of the class

- Java adds an implicit object reference `this.` to refer to the current object
  - Below, `isEven(8)` is really `this.isEven(8)`

```
boolean isEven(int n) {
    return n % 2 == 0;
}

void main() {
    IO.println(isEven(8));
}
```

## Pause and predict

**Q.** Which line does **NOT** compile?

- **A.** `"abc".length()`
- **B.** `Math.pow(2, 5)`
- **C.** `String.substring(1, 4)`
- **D.** `Integer.parseInt("4")`

# ArrayList: Types

## `ArrayList` Types

**Type inference:** we don't need to repeat the type:

```
ArrayList<String> a = new ArrayList<String>();    // explicit
ArrayList<String> b = new ArrayList<>();          // "diamond:" the right side is infe
var               c = new ArrayList<String>();    // var: the left side is inferred
```

**No primitives.** `ArrayList` can hold objects only. `ArrayList<int>` is illegal.

The workaround: Use the **wrapper** class:

```
void main() {
    ArrayList<Integer> scores = new ArrayList<>();
    scores.add(95);                // int 95 auto-boxed to Integer
    int first = scores.get(0);     // auto-unboxed back to int

    IO.println(first);
}
```

- In line 2, Java converts ("auto-boxes") the `int` to an `Integer` for us
- In line 3, Java converts ("auto-unboxes") the `Integer` back to an `int`

# ArrayList: Methods

## `set(i, e)` vs `add(i, e)`

Watch out: **different behavior**:

```
void main() {
    var xs = new ArrayList<String>();
    xs.add("x"); xs.add("y"); xs.add("z");     // [x, y, z]

    // REPLACE index 0; returns the old value
    String old = xs.set(0, "X");
    // old is "x",  xs is now [X, y, z]

    // INSERT at index 0; shifts the rest right
    xs.add(0, "W");
    // xs is now [W, X, y, z]
}
```

- `set(i, e)` **replaces** and **returns the old value**.
  - Valid only for `0 ≤ i < size()`
  - It can't append.
- `add(i, e)` **inserts** and grows the list, pushing everything from `i` onward one slot right.

## Useful `ArrayList` methods



Reference material

| Method | What it does |
|---|---|
| `add(e)` | append `e` to the **end** |
| `add(i, e)` | insert `e` at index `i`, **pushing the rest down** |
| `get(i)` | return the element at index `i` |
| `set(i, e)` | **replace** the element at index `i` with `e` |
| `remove(e)` | remove the first occurrence of `e` (shifting the rest up) |
| `contains(e)` | `true` if `e` is in the list |

| | |
|---|---|
| `size()` | the current number of elements |

Three different "how big" words:

- `s.length()` for a **String**,
- `a.length` for an **array**,
- `list.size()` for an **`ArrayList`**.

## Pause and predict

```
var xs = new ArrayList<String>();
xs.add("a"); xs.add("b"); xs.add("c");
xs.set(1, "B");
xs.add(1, "X");
IO.println(xs);
```

What prints?

- **A.** `[a, B, X, c]`
- **B.** `[a, X, B, c]`
- **C.** `[a, X, c]`
- **D.** `[a, B, c]`

## Demo: Word List with ArrayList

**Problem:** Refactor Monday's `WordList` so the words live in an `ArrayList<String>` instead of a fixed `String[]`.

We will work on `WordListOld.java` in class.

The file `WordList.java` shows the finished product. Each `// was:` comment marks a line we changed:

- `String[] words; int size;` → `ArrayList<String> words;` : the list knows its size.
- `new String[MAXWORDS]` → `new ArrayList<>()` : no guessing at the capacity
- `words[size++] = w` → `words.add(w)` : no size field to maintain by hand
- `words[i]` → `words.get(i)`, and `size` → `words.size()`.

The histogram is still a plain `int[]` . Its size (50) is a known bound, so an array is the right choice here. (You could still use an ArrayList if you wanted, but you would have to pre-fill it with zeroes.)

Press **Run**. Same numbers, one fewer thing to get wrong.

# Scanner

## Scanner: reading input

So far we've read our input with `IO.readln()`.

This is a great way to get started, but it is more limited than other (more complicated) approaches.

Most of the Java world uses **`Scanner`** or used it in code already written. We must be able to read it.

## Imports

The classes we meet today are not available by default.

We must use **import** statements at the top of our file

```
import java.util.Scanner;
import java.util.ArrayList;
```



The programs we have been writing in class are **compact source files:**

No "class" definition, just methods

These files automatically import most of the classes you might use, so you can just use Scanner or ArrayList there.

Once you start writing "real" classes, **import statements become mandatory.**

To use Scanner methods, we need to create an object:

```
Scanner in = new Scanner(System.in);
```

- Scanner is  a kind-of-a-thing
  - Each individual Scanner object has its own input source (a file, a String, the keyboard)

- We use `new` but we place the input source inside the parentheses
  - `System.in` means the keyboard (standard input).
  - many more examples later...

Basic usage: the next... methods:

```
int    age   = in.nextInt();
double price = in.nextDouble();
String line  = in.nextLine();
```



**Warning:** only create **one** Scanner on System.in per program!

You must use either IO.readln OR Scanner(System.in) but you **cannot use both** in the same program**!**

# Scanner Methods

## Scanner's methods

There are many methods available:

| Call | Reads and returns |
|---|---|
| `in.nextInt()` | the next `int` |
| `in.nextDouble()` | the next `double` |
| `in.next()` | the next **word** (up to whitespace) |
| `in.nextLine()` | the **rest of the line** (a whole `String`) |
| `in.hasNextInt()` | `true` if another `int` is waiting (peek, don't read) |
| `in.hasNext()` | `true` if another word is waiting (just peek) |
| `in.hasNextLine()` | `true` if another line is waiting (just peek) |

- `nextInt()` gives you a **number directly**
  - no `Integer.parseInt` needed
  - also crashes your program if a number isn't next

- The hasNext... methods are used to check whether the corresponding next... method will succeed.

- When reading from System.in all of these methods will sit and wait for the user to enter input

## Scanner in action

Here's a textbook style example of classic Java:

```
void main() {
    Scanner in = new Scanner(System.in);
    System.out.print("Enter two integers: ");
    int x = in.nextInt();
    int y = in.nextInt();
    System.out.println("sum = " + (x + y));
}
```

## Side-by-side Comparison

| | Modern style (JDK 25+) | Classic dialect (`Scanner`) |
|---|---|---|
| import | none | `import java.util.Scanner;` |
| set up | nothing | `Scanner in = new Scanner(System.in);` |
| read an int | `int x = Integer.parseInt(IO.readln());` | `int x = in.nextInt();` |
| read a line | `String s = IO.readln();` | `String s = in.nextLine();` |

Both are correct.

Scanner has one big advantage: we can peek ahead to see if the data actually is an integer or double:

```
void main() {
    Scanner in = new Scanner(System.in);
    IO.print("Enter an integer: ");
    if (in.hasNextInt()) {              // peek: is an int waiting?
        int n = in.nextInt();
        IO.println("Got " + n);
    } else {
        IO.println("That wasn't an integer: " + in.next());
    }
}
```

# The Scanner Trap

## Predict: the infamous Scanner trap

```
Scanner in = new Scanner(System.in);
System.out.print("Age? ");
int age = in.nextInt();
System.out.print("Name? ");
String name = in.nextLine();     // read the name
System.out.println("age=[" + age + "] name=[" + name + "]");
```

The user types `25`, Enter, then `Alice`, Enter.

### Pause and predict

**Q.** The user types `25`, Enter, then `Alice`, Enter. What does `name` end up as?

- **A.** `"Alice"`
- **B.** `""` the empty string
- **C.** `"25"`
- **D.** Crash

```
void main() {
    Scanner in = new Scanner(System.in);
    IO.print("Age? ");
    int age = in.nextInt();
    IO.print("Name? ");
    String name = in.nextLine();     // read the name
    IO.println("age=[" + age + "] name=[" + name + "]");
}
```

Let's draw it out. We can think of keyboard input as a pipe holding input waiting to be read.

The trap only bites when you try to read a line after a smaller unit:

```
void main() {
    Scanner in = new Scanner(System.in);
    System.out.print("Enter an integer: ");
    int x = in.nextInt();
    System.out.print("Enter another integer: ");
    int y = in.nextInt();
    System.out.println("sum = " + (x + y));
}
```

After reading 2, the pipe looks like `\n3\n` and `nextInt` will read past the newline.

# Scanner Trap: Solution

## Why the trap happens (and the fix)

`nextInt()` reads the `25` but **leaves the Enter keypress (newline) sitting in the buffer**.

The next `nextLine()` reads *from the cursor to that newline*

Results: an empty string.

Remember: nextLine() will remove the Enter, but nextInt() and next() **do not**

```
buffer after nextInt():  25⏎Alice⏎
                           ↑ cursor here — nextLine() reads "" up to ⏎
```

- Diagnosis: When you test you notice a prompt gets skipped right after reading a number.
- Fix**:** add an extra `in.nextLine()` after `in.nextInt()` to consume the leftover newline.
- JDK 25+'s `IO.readln` dodges this,
    - but at the cost that we can't peek ahead to see if a valid `int` is waiting

# Random

## Random: making the computer roll dice

Why do we need random numbers?

- Games,
- simulations,
- sampling,
- test data

Use a `Random` object:

```
void main() {
    Random rng = new Random();      // make one Random object
    int roll = rng.nextInt(6);      // an int in [0, 6): 0,1,2,3,4,5

    IO.println(roll);
}
```

`new Random()` builds the object.

Then you ask it for numbers with its methods.

# Random Methods

## Random's methods

| Call | Returns |
|---|---|
| `rng.nextInt(n)` | an `int` in **`[0, n)`**<br>That is: `0` up to `n-1` |
| `rng.nextInt()` | any `int`, positive or negative |
| `rng.nextDouble()` | a `double` in **`[0.0, 1.0)`** |
| `rng.nextBoolean()` | `true` or `false` |

**Trick: Shifting the range**

- `nextInt(n)` starts at 0, so add an offset for other ranges:

```
int die   = 1 + rng.nextInt(6);      // [1, 6]  a six-sided die
int score = 50 + rng.nextInt(51);    // [50, 100] inclusive
```

## Seeding: the *same* randomness every run

Pass a **seed** to the constructor and you get the *same sequence* every time:

```
void main() {
    Random rng = new Random(42);
    for (int i = 0; i < 10; i++)
        IO.print((1 + rng.nextInt(6)) + " ");
}
```

```
run 1:  3 4 1 3 1 2 6 3 2 6
run 2:  3 4 1 3 1 2 6 3 2 6
```

This is a **feature**, not a bug.

Reproducible runs are what make testing and debugging possible.

## Pause and predict

```
Random rng = new Random();
int die = 1 + rng.nextInt(1);
```

What values can `die` take?

- **A.** 0 only
- **B.** 1 only
- **C.** 2 only
- **D.** 0 through 1
- **E.** 1 through 2

## Conclusion

### what's *inside* an object?

You've now called methods on `Scanner`, `Random`, `ArrayList`, `String`, `Math`. Every one of those:

```
list.add("cheese");      // someone wrote the add method
in.nextInt();            // someone wrote the nextInt method
```

has

- **fields** (its data) and
- **methods** (its operations) that

that *someone wrote*.

**Next week:** you'll write your own classes.

## Take Home: Grade Book

An `ArrayList<Integer>` of exam scores. Implement three methods:

- `double average(ArrayList<Integer> scores)` : the mean, as a real number. An **empty** list has average `0.0` (don't crash, don't divide by zero).
- `int countAbove(ArrayList<Integer> scores, int cutoff)` : how many scores are **strictly** greater than `cutoff`.
- `ArrayList<Integer> above(ArrayList<Integer> scores, int cutoff)` : a **new** list holding just the scores strictly above `cutoff`, in their original order. The list you were given must not change.

Remember: `size()` for an `ArrayList` and elements go in with `add(e)` and come out with `get(i)`. Java boxes and unboxes the `int`s for you.

Press **Test** to run the tests.